Formula 1 World Championship Database: A PostgreSQL System for Motorsport Analytics (1950–2024)

Sourabh Raja
University at Buffalo
sourabhr

Sphoorthy Selvaraj
University at Buffalo
selvara3

Sindhu Shrishailappa Jumanal
University at Buffalo
sjumanal

TABLE I
PROJECT INFORMATION

Field	Value
Title	F1 Race Analytics Database
Course	CSE 560 — Data Models and Query Languages
Term	Spring 2026, University at Buffalo
Phase	Phase 1 — Project Proposal
Due Date	March 15, 2026
Dataset	Formula 1 World Championship (Kaggle)
DBMS	PostgreSQL
Records	700,000+ across 14 tables
Coverage	1950–2024 (75 seasons)

***Abstract*—This report presents the Phase 1 proposal for the F1 Race Analytics Database, a PostgreSQL-based relational database built on the Formula 1 World Championship dataset spanning 75 years of racing history (1950–2024). The system integrates 14 interrelated tables covering circuits, drivers, constructors, race results, lap times, pit stops, qualifying sessions, and championship standings, totalling over 700,000 records. This proposal covers the problem domain, target users, entity-relationship design, relational schema, and a complete BCNF normalization analysis verified directly from the source CSV data.**

I. PROJECT DETAILS

Project: Formula 1 World Championship Database
Course: CSE 560 – Data Models and Query Language, Spring 2026
Institution: University at Buffalo, CSE Department
Instructor: Prof. Shamsad Parvin, PhD

A. Project Information

Table I summarizes the key project metadata.

B. Team Members and Contribution

Table II lists all team members, their UB IDs, and contribution. Each member contributed equally to all parts of the project.

TABLE II
TEAM CONTRIBUTION SUMMARY

Name	Primary Role	%
Sphoorthy Selvaraj	Task 1, Task 2 Schema Design	34%
Sourabh Raja	Task 3 Data Acquisition, SQL	33%
Sindhu S Jumanal	Task 4 Normalization, E/R Diagram	33%

II. PROBLEM STATEMENT

Formula 1 appealed to us as a project domain because it sits at a genuinely interesting intersection of engineering, statistics, and sports history. The data is highly relational by nature, races connect to circuits, drivers connect to constructors, lap times connect to both races and drivers and it spans 74 years, which makes trend analysis meaningful rather than just a point-in-time snapshot.

A. Title and Core Problem

Title: *F1 Race Analytics Database: A PostgreSQL System for Historical Formula 1 Performance Analysis*

The problem lies in the absence of a unified, queryable relational system over the complete F1 historical record. Existing approaches rely on spreadsheets that cannot support multi-table joins, enforce referential integrity, or handle the data volume involved. This database which we propose is designed to answer the following questions:

- Which driver has the highest average points-per-race over a full career?
- Which constructor showed the greatest improvement in standings between any two consecutive seasons?
- How does pit stop duration correlate with final finishing position?
- How have fastest lap times at circuits ?
- What is the statistical relationship between qualifying position and race result?

B. Background

The F1 dataset used in this project contains 14 interrelated CSV files covering every race from 1950 to 2024. The data spans over multiple entity types such as drivers, constructors, circuits, race events, lap times, pit stops, and standings with

TABLE III
TARGET USER GROUPS

User	Access	Primary Use
F1 Analyst	Read-only	Career stats, lap time trends
Journalist	Read-only	Historical comparisons, race reports
Race Strategist	Read/Write	Pit stop and qualifying analysis
DBA	Full	Data loading, schema, maintenance

complex many-to-many relationships between them. A spreadsheet cannot represent these relationships without redundancy, update anomalies and it is is cumbersome to fetch results from it.

There is a significant problem without the database. First, F1 analytics is an active domain used by teams, journalists, and broadcasters to derive competitive insights. Second, the dataset scale (700,000+ records) exceeds what any spreadsheet tool can manage efficiently. Third, the multi-entity nature of the data, a single race involves drivers, constructors, a circuit, lap times per driver per lap, pit stops, and a result record , demands relational modelling with enforced foreign key constraints and normalization.

C. Why a Database Instead of Excel

A spreadsheet tool such as Microsoft Excel is fundamentally unsuitable for this use case for the following reasons:

The dataset has 589,081 lap time records alone. A spreadsheet cannot handle that scale, cannot enforce referential integrity across 14 linked tables, and cannot execute the kind of multi-table transaction that inserting a complete race weekend requires. PostgreSQL gives us foreign-key constraints, ACID transactions, stored procedures, triggers, and indexed lookups

D. Contribution and Potential

This project delivers a fully normalized, query-ready PostgreSQL database over the complete F1 historical record. Its contributions to the domain are:

- A verified BCNF-compliant schema for all 14 tables, with functional dependencies derived directly from the source data.
- A reusable SQL foundation for F1 analytics that supports both historical queries and post-race updates.
- Demonstration of advanced SQL features including window functions, correlated subqueries, stored procedures and triggers.

III. TARGET USERS

The database serves four user groups with different access needs, summarized in Table III.

A. Primary Users

F1 Analysts query multi-table joins across `lap_times`, `results`, and `circuits` to compare driver performance across track types and seasons.

Sports Journalists retrieve head-to-head statistics and historical records during race weekends using aggregation queries on `results` and `driver_standings`.

B. Secondary Users

Race Strategists query `pit_stops` and `lap_times` during live races to evaluate undercut opportunities, with limited write access for session notes.

IV. TASK 2: SCHEMA DESIGN

A. E/R Diagram

Figure 1 shows the entity-relationship diagram. Tables with orange-red title bars are the two that contain BCNF violations in the initial schema (discussed in Task 4). Double-bordered tables are weak entities with composite primary keys.

The schema has four layers. DRIVER, CONSTRUCTOR, CIRCUIT, and SEASON are master entities that change rarely and are referenced by everything else. RACE and RESULT form the central table where most queries land. LAP_TIME, PIT_STOP, QUALIFYING, and SPRINT_RESULT provide granular per-event detail. The standings tables track championship progress race by race.

B. Relational Schema

All 14 tables are described below. Column widths in the attribute tables follow the convention: **PK** = primary key in red, FK = foreign key in blue.

1) SEASONS — 75 rows*:*

Attribute	Type	Null	Notes
year	INT	No	**PK**; 1950–2024
url	TEXT	Yes	Wikipedia season link

Year is the primary key.

2) CIRCUITS — 77 rows*:*

Attribute	Type	Null	Notes
circuitId	SERIAL	No	**PK**
circuitRef	VARCHAR(50)	No	Candidate key; URL slug
name	VARCHAR(100)	No	Candidate key; circuit name
location	VARCHAR(100)	Yes	City or region
country	VARCHAR(60)	Yes	Country name
lat	NUMERIC(8,5)	Yes	Latitude coordinate
lng	NUMERIC(8,5)	Yes	Longitude coordinate
alt	INT	Yes	Altitude in metres
url	TEXT	Yes	Wikipedia circuit link

Three candidate keys are verified: circuitId, circuitRef, and name are each unique across all 77 rows. FK action: circuitId is referenced by RACE with RESTRICT on delete.

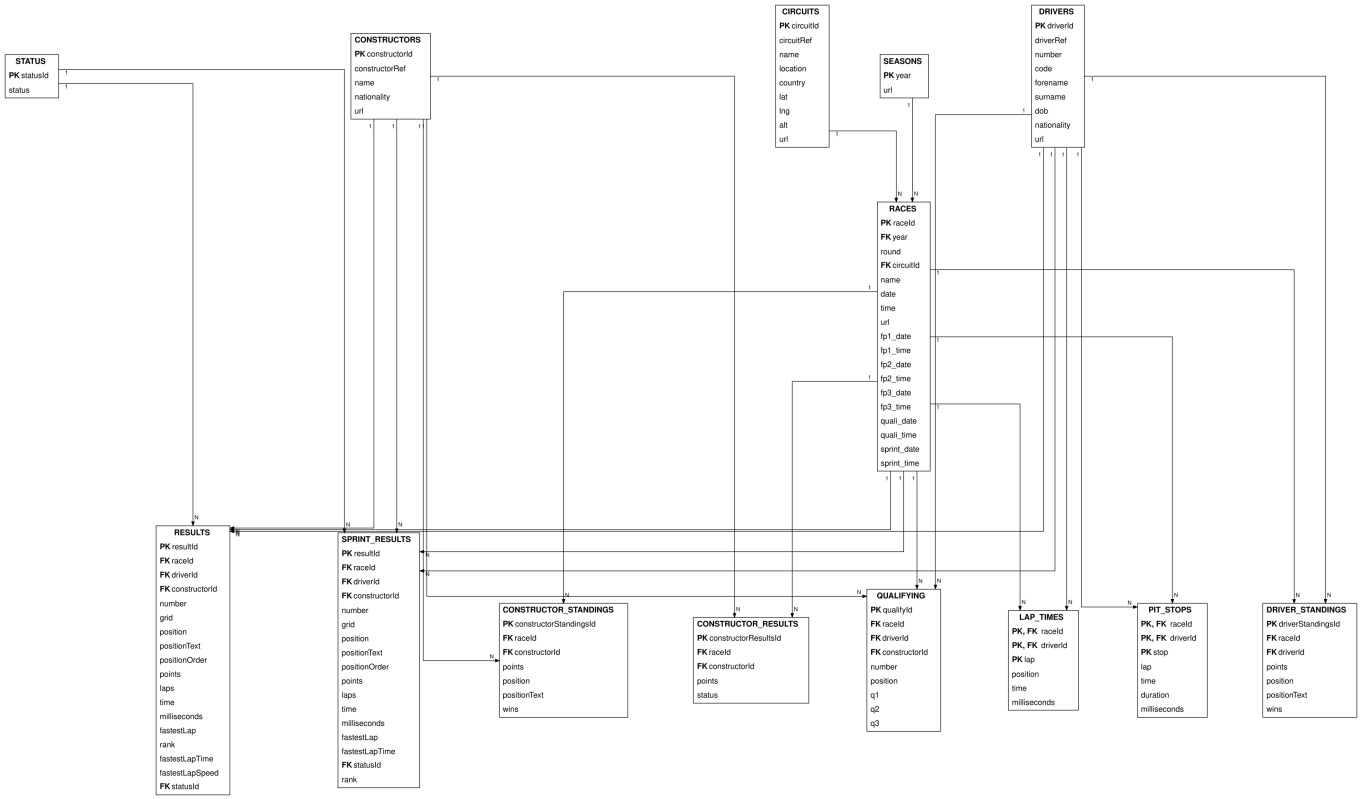


Fig. 1. F1 Database E/R Diagram.

3) CONSTRUCTORS — 212 rows*:*

Attribute	Type	Null	Notes
constructorId	SERIAL	No	**PK**
constructorRef	VARCHAR(50)	No	Candidate key; URL slug
name	VARCHAR(100)	No	Candidate key; team name
nationality	VARCHAR(50)	Yes	e.g., British (86 teams)
url	TEXT	Yes	Wikipedia team link

Three candidate keys: constructorId, constructorRef, name. FK action: constructorId is referenced by RESULT, QUALIFYING, and the standings tables with CASCADE on delete.

4) DRIVERS — 861 rows*:*

Attribute	Type	Null	Notes
driverId	SERIAL	No	**PK**
driverRef	VARCHAR(50)	No	Candidate key; URL slug
number	INT	Yes	Perm. number; 802/861 null
code	CHAR(3)	Yes	NOT a CK; 7 codes reused
forename	VARCHAR(50)	No	First name
surname	VARCHAR(50)	No	Surname
dob	DATE	Yes	Date of birth
nationality	VARCHAR(50)	Yes	e.g., British (166 drivers)
url	TEXT	Yes	Wikipedia driver link

Only driverId and driverRef are candidate keys. The code column looks like a unique identifier but seven codes (ALB, MSC, MAG, DOO, VER, BIA, HAR) were reused by different drivers across different seasons. FK action: driverId is referenced by RESULT, LAP_TIME, PIT_STOP, QUALIFYING.

5) STATUS — 139 rows*:*

Attribute	Type	Null	Notes
statusId	SERIAL	No	**PK**
status	VARCHAR(50)	No	Candidate key; e.g., Finished

FK : statusId referenced by RESULT and SPRINT_RESULT with RESTRICT on delete.

6) RACES — 1,125 rows*:*

Attribute	Type	Null	Notes
raceId	SERIAL	No	**PK**
year	INT	No	FK → SEASONS; RESTRICT
round	INT	No	Round number in season
circuitId	INT	No	FK → CIRCUITS; RESTRICT
name	VARCHAR(100)	No	Grand Prix name
date	DATE	No	Race date
time	TIME	Yes	Race start time
url	TEXT	Yes	Wikipedia race link
fp1_date	DATE	Yes	Practice 1 date
fp2_date	DATE	Yes	Practice 2 date
fp3_date	DATE	Yes	Practice 3 date
quali_date	DATE	Yes	Qualifying date
sprint_date	DATE	Yes	Sprint race date

The composite {year, round} is a candidate key.

7) RESULTS — 26,759 rows*:*

Attribute	Type	Null	Notes
resultId	SERIAL	No	**PK** (only true key)
raceId	INT	No	FK → RACES; CASCADE
driverId	INT	No	FK → DRIVERS; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
number	INT	Yes	Car number used this race
grid	INT	Yes	Starting grid position
position	INT	Yes	Finish position; 41% null
positionText	VARCHAR(5)	Yes	Text version of position
positionOrder	INT	No	Always-populated sort field
points	NUMERIC(5,2)	No	Championship points scored
laps	INT	Yes	Laps completed
time	VARCHAR(20)	Yes	Finish time string
milliseconds	INT	Yes	Finish time in ms
fastestLap	INT	Yes	Lap number of fastest lap
rank	INT	Yes	Fastest lap rank in race
fastestLapTime	VARCHAR(10)	Yes	e.g., 1:27.452
fastestLapSpeed	NUMERIC(7,3)	Yes	Speed in km/h
statusId	INT	No	FK → STATUS; RESTRICT

resultId is the only valid primary key.

8) QUALIFYING — 10,494 rows*:*

Attribute	Type	Null	Notes
qualifyId	SERIAL	No	**PK**
raceId	INT	No	FK → RACES; CASCADE
driverId	INT	No	FK → DRIVERS; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
number	INT	Yes	Car number used
position	INT	Yes	Qualifying result
q1	VARCHAR(10)	Yes	Q1 lap time
q2	VARCHAR(10)	Yes	Q2 time
q3	VARCHAR(10)	Yes	Q3 time

{raceId, driverId} IS a candidate key here

9) LAP_TIMES — 589,081 rows (weak entity)*:*

Attribute	Type	Null	Notes
raceId	INT	No	**PK** + FK → RACES; CASCADE
driverId	INT	No	**PK** + FK → DRIVERS; CASCADE
lap	INT	No	**PK**; lap number
position	INT	Yes	Track position end of lap
time	VARCHAR(10)	Yes	Lap time, e.g., 1:27.452
milliseconds	INT	Yes	Same time in ms

Composite PK {raceId, driverId, lap} is unique across all rows

10) PIT_STOPS — 11,371 rows*:*

Attribute	Type	Null	Notes
raceId	INT	No	**PK** + FK → RACES; CASCADE
driverId	INT	No	**PK** + FK → DRIVERS; CASCADE
stop	INT	No	**PK**; pit stop number
lap	INT	No	Lap on which stop occurred
time	TIME	Yes	Time of day of stop
duration	VARCHAR(10)	Yes	Duration string
milliseconds	INT	Yes	Duration in ms

Composite PK {raceId, driverId, stop} verified across all rows

11) DRIVER_STANDINGS — 34,863 rows*:*

Attribute	Type	Null	Notes
driverStandId	SERIAL	No	**PK**
driverStandingsId	SERIAL		
raceId	INT	No	FK → RACES; CASCADE
driverId	INT	No	FK → DRIVERS; CASCADE
points	NUMERIC(6,2)	No	Cumulative points
position	INT	Yes	Championship position
wins	INT	No	Cumulative wins
is_disqualified	BOOLEAN	No	Default FALSE; see Task 4

Note: positionText was removed during BCNF decomposition and replaced with `is_disqualified`*.*

12) CONSTRUCTOR_STANDINGS — 13,391 rows*:*

Attribute	Type	Null	Notes
constrStandId*	SERIAL	No	**PK**
raceId	INT	No	FK → RACES; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
points	NUMERIC(6,2)	No	Cumulative points
position	INT	Yes	Championship position
wins	INT	No	Cumulative wins
is_excluded	BOOLEAN	No	Default FALSE; see Task 4

Note: positionText removed during BCNF decomposition. Replaced with `is_excluded`*.*

13) SPRINT_RESULTS — 360 rows*:*

Attribute	Type	Null	Notes
resultId	SERIAL	No	**PK**
raceId	INT	No	FK → RACES; CASCADE
driverId	INT	No	FK → DRIVERS; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
position	INT	Yes	Sprint finish position
positionText	VARCHAR(5)	Yes	Text version
points	NUMERIC(4,2)	No	Max 8 points for sprint win
laps	INT	Yes	Laps completed
statusId	INT	No	FK → STATUS; RESTRICT

{raceId, driverId} IS a candidate key here (360 unique pairs verified). Unlike the main results table, no constructor swaps occur in sprint races. 18 sprint events from 2021–2024.

14) CONSTRUCTOR_RESULTS — 12,625 rows*:*

Attribute	Type	Null	Notes
constrResultsId	SERIAL	No	**PK**
raceId	INT	No	FK → RACES; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
points	NUMERIC(5,2)	Yes	Points this race
status	CHAR(1)	Yes	'D' = disqualified; else null

V. Task 3: Dataset Acquisition

A. Source and Access

The data comes from the Ergast Motor Racing Developer API [1], which has maintained Formula 1 records since 2005. Rohan Rao packaged the full Ergast database as 14 CSV files on Kaggle, updated through January 2025 [2].

Download link: https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020

No account or special access is required. The dataset is published under an open licence for non-commercial use.

B. BCNF Analysis

For each relation, we verify that every non-trivial functional dependency $X \to Y$ has X as a superkey. This is confirmed by computing X^+ from the source data.

1) seasons:

- **Attributes:** {year, url}
- **FDs:** $year \to url$, $url \to year$
- **Closures:** $\{year\}^+ = R$, $\{url\}^+ = R$
- **Candidate Keys:** {year}, {url}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

2) circuits:

- **Attributes:** {circuitId, circuitRef, name, location, country, lat, lng, alt, url}
- **FDs:** $circuitId \to all$, $circuitRef \to all$, $name \to all$
- **Closures:** $\{circuitId\}^+ = R$, $\{circuitRef\}^+ = R$, $\{name\}^+ = R$
- **Candidate Keys:** {circuitId}, {circuitRef}, {name}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

3) drivers:

- **Attributes:** {driverId, driverRef, number, code, forename, surname, dob, nationality, url}
- **FDs:** $driverId \to all$, $driverRef \to all$
- **Closures:** $\{driverId\}^+ = R$, $\{driverRef\}^+ = R$
- **Candidate Keys:** {driverId}, {driverRef}
- **Note:** code and number are not candidate keys — 7 codes are reused across eras, and 802/861 numbers are NULL.
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

4) constructors:

- **Attributes:** {constructorId, constructorRef, name, nationality, url}
- **FDs:** $constructorId \to all$, $constructorRef \to all$, $name \to all$
- **Closures:** $\{constructorId\}^+ = R$, $\{constructorRef\}^+ = R$, $\{name\}^+ = R$
- **Candidate Keys:** {constructorId}, {constructorRef}, {name}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

5) status:

- **Attributes:** {statusId, status}
- **FDs:** $statusId \to status$, $status \to statusId$
- **Closures:** $\{statusId\}^+ = R$, $\{status\}^+ = R$
- **Candidate Keys:** {statusId}, {status}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

6) races:

- **Attributes:** {raceId, year, round, circuitId, name, date, time, url, fp1_date, fp2_date, fp3_date, quali_date, sprint_date}
- **FDs:** $raceId \to all$, $(year, round) \to all$
- **Closures:** $\{raceId\}^+ = R$, $\{year, round\}^+ = R$
- **Candidate Keys:** {raceId}, {year, round}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

7) results:

- **Attributes:** {resultId, raceId, driverId, constructorId, grid, position, positionText, positionOrder, points, laps, time, milliseconds, fastestLap, rank, fastestLapTime, fastestLapSpeed, statusId}
- **FDs:** $resultId \to all$
- **Closures:** $\{resultId\}^+ = R$
- **Candidate Keys:** {resultId} only
- **Note:** $(raceId, driverId)$ is not a candidate key , 176 rows exist where a driver raced under two constructors in the same race.
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

8) qualifying:

- **Attributes:** {qualifyId, raceId, driverId, constructorId, number, position, q1, q2, q3}
- **FDs:** $qualifyId \to all$, $(raceId, driverId) \to all$
- **Closures:** $\{qualifyId\}^+ = R$, $\{raceId, driverId\}^+ = R$
- **Candidate Keys:** {qualifyId}, {raceId, driverId}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

9) lap_times:

- **Attributes:** {raceId, driverId, lap, position, time, milliseconds}
- **FDs:** $(raceId, driverId, lap) \to position, time, milliseconds$

- **Closures:** $\{raceId, driverId, lap\}^+ = R$
- **Candidate Keys:** {raceId, driverId, lap}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

10) pit_stops:

- **Attributes:** {raceId, driverId, stop, lap, time, duration, milliseconds}
- **FDs:** $(raceId, driverId, stop) \rightarrow lap, time, duration, milliseconds$
- **Closures:** $\{raceId, driverId, stop\}^+ = R$
- **Candidate Keys:** {raceId, driverId, stop}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

11) constructor_results:

- **Attributes:** {constructorResultsId, raceId, constructorId, points, status}
- **FDs:** $constructorResultsId \rightarrow all$, $(raceId, constructorId) \rightarrow all$
- **Closures:** $\{constructorResultsId\}^+ = R$, $\{raceId, constructorId\}^+ = R$
- **Candidate Keys:** {constructorResultsId}, {raceId, constructorId}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

12) sprint_results:

- **Attributes:** {resultId, raceId, driverId, constructorId, position, positionText, points, laps, statusId}
- **FDs:** $resultId \rightarrow all$, $(raceId, driverId) \rightarrow all$
- **Closures:** $\{resultId\}^+ = R$, $\{raceId, driverId\}^+ = R$
- **Candidate Keys:** {resultId}, {raceId, driverId}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

13) driver_standings — BCNF Violation:

- **Attributes:** {driverStandingsId, raceId, driverId, points, position, positionText, wins}
- **FDs:** $driverStandingsId \rightarrow all$, $(raceId, driverId) \rightarrow all$, $position \rightarrow positionText$
- **Closure of violating determinant:** $\{position\}^+ = \{position, positionText\} \neq R$
- **Candidate Keys:** {driverStandingsId}, {raceId, driverId}
- **Violation:** $position \rightarrow positionText$ violates BCNF as position is not a superkey.

Resolution: `positionText` is dropped and replaced with `is_disqualified` BOOLEAN (default FALSE), capturing the only meaningful non-numeric case. The resulting table is in BCNF as all remaining FDs have a superkey on the LHS.

14) constructor_standings — BCNF Violation:

- **Attributes:** {constructorStandingsId, raceId, constructorId, points, position, positionText, wins}
- **FDs:** $constructorStandingsId \rightarrow all$, $(raceId, constructorId) \rightarrow all$, $position \rightarrow positionText$
- **Closure of violating determinant:** $\{position\}^+ = \{position, positionText\} \neq R$
- **Candidate Keys:** {constructorStandingsId}, {raceId, constructorId}
- **Violation:** $position \rightarrow positionText$ violates BCNF as position is not a superkey. Verified: 17 rows have positionText = 'E' (Excluded) while position holds a numeric value.

Resolution: `positionText` is dropped and replaced with `is_excluded` BOOLEAN (default FALSE), capturing the only meaningful non-numeric case. The resulting table is in BCNF as all remaining FDs have a superkey on the LHS.

Final schema:

```
DRIVER_STANDINGS(driverStandingsId, raceId,
    driverId, points, position, wins,
        blueis_disqualified)
```

FDs after decomposition: $\{driverStandingsId\} \rightarrow \{all\}$; $\{raceId, driverId\} \rightarrow \{all\}$. Both are superkeys. **BCNF confirmed.**

Lossless: positionText is exactly reconstructable from `position` and `is_disqualified`. No information is lost.

15) CONSTRUCTOR_STANDINGS:

Violation: $\{position\} \rightarrow \{positionText\}$ fails because position = 11 maps to both "11" (normal) and "E" (Excluded) in 17 rows for constructorId = 1. Same structural problem as driver standings.

Initial schema:

```
CONSTRUCTOR_STANDINGS(constructorStandingsId,
  raceId, constructorId, points, position,
          redpositionText, wins)
```

Decomposition: Remove `positionText`. Add `is_excluded BOOLEAN DEFAULT FALSE`. Reconstructable as:

```
CASE WHEN is_excluded THEN 'E' ELSE CAST(position AS
                VARCHAR) END
```

Final schema:

```
CONSTRUCTOR_STANDINGS(constructorStandingsId,
raceId, constructorId, points, position, wins,
              blueis_excluded)
```

FDs after decomposition: $\{constructorStandingsId\} \rightarrow \{all\}$; $\{raceId, constructorId\} \rightarrow \{all\}$. **BCNF confirmed.**
Lossless: positionText reconstructable from `position` and `is_excluded`. No information lost.

C. Final Schema BCNF Summary

After decomposition, all 14 relations satisfy BCNF. Twelve of fourteen tables were already in BCNF. The two violations shared the same root cause and were both resolved through lossless decomposition that preserves all original information.

VI. Conclusion

This report covers Phase 1 of our Formula 1 World Championship Database. The domain was chosen because the data is interesting, freely available, and structured in a way that naturally maps to a relational schema. Most of the schema design followed directly from the 14 source files, with careful attention to which composite keys were genuine candidates and which only appeared to be. The normalization analysis was the most instructive part of the work - checking the actual data rows turned up findings that a theory-only approach would have missed.

References

[1] Ergast Developer API, "Motor Racing Data," http://ergast.com/mrd/, accessed 2025.

[2] R. Rao, "Formula 1 World Championship (1950–2024)," Kaggle, https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020, updated Jan. 2025.

[3] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed., McGraw-Hill, 2003.

[4] R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.

[5] IEEE, "Conference Paper Template," https://www.ieee.org/conferences/publishing/templates.html, 2024.